



Deep Learning II

Project Report

Deep Neural Networks for Handwritten Digit Classification

Produced by :

ZARGUI RAYEN

Supervised by :

YOHAN PETETIN

M2 Data Science



GitHub Repository : [Generative-pretraining-mnist](#)

2025/2026

Table des matières

Introduction	3
Implemented Models	4
1.1 Restricted Boltzmann Machines	4
1.2 Deep Belief Networks	5
1.3 Definition of Deep Neural Network (DNN)	6
Introduction to Datasets	7
2.1 Binary AlphaDigits Dataset	7
2.2 MNIST Dataset	7
Experimental Protocol	8
3.1 Execution Environment	8
3.2 Training Configuration	8
3.3 Architectures Explored	8
3.4 Evaluation Methodology	9
3.5 How to Run the Code	9
Testing the Performance of Generative Models on the AlphaDigits Dataset	10
4.1 Restricted Boltzmann Machine on Binary AlphaDigits	10
4.1.1 Generation and reconstruction on the letter A	10
4.1.2 Effect of model capacity	11
4.1.3 Effect of the learning rate	11
4.1.4 Joint hyperparameter inspection	12
4.2 Deep Belief Network on Binary AlphaDigits	12
4.2.1 Layer-wise pretraining and generation	13
4.2.2 Effect of DBN depth	13
4.3 Influence of the Number of Learned Classes	14
4.3.1 RBM	14
4.3.2 DBN	14
Application of Generative Models for Pretraining Deep Neural Networks on the MNIST Dataset	15
5.1 Classification Pipeline	15
5.2 Effect of Pretraining on Optimization	15
5.3 Influence of Depth, Width, and Training-Set Size	16
5.3.1 Effect of Depth	16
5.3.2 Effect of Width	17
5.3.3 Effect of Training-Set Size	17
5.4 Qualitative Analysis of the Pretrained Classifier	18
5.4.1 Softmax Confidence	18
5.4.2 Misclassified Examples	18
5.4.3 Confusion Matrix	19
5.5 Best Final Configuration	19

Bonus : Testing the Performance of Generative Models on the MNIST Dataset	20
6.1 Global Visual Comparison	20
6.2 RBM	21
6.3 DBN	21
6.4 VAE	22
6.5 GAN	22
6.6 DDPM	23
Conclusion	24

Introduction

Deep learning has become a central tool in artificial intelligence because it allows models to learn rich and hierarchical representations directly from data. In image classification in particular, deep neural networks (DNNs) have achieved strong performance. However, training deep architectures can still be difficult, especially when the initialization is poor, the network is deep, or the amount of labeled data is limited.

One classical way to address this issue is *unsupervised pretraining*. Instead of initializing all hidden layers randomly, the network is first trained in an unsupervised manner to learn useful intermediate features, and is then fine-tuned with supervised backpropagation. This idea is closely connected to *Restricted Boltzmann Machines* (RBMs) and *Deep Belief Networks* (DBNs), which were introduced as generative models for representation learning.

The goal of this project is to study the effect of such pretraining on deep neural network performance. We compare two identical DNNs for handwritten digit classification : one trained from scratch, and one initialized through RBM/DBN pretraining before supervised fine-tuning. To do so, we first validate RBMs and DBNs on the Binary AlphaDigits dataset through reconstruction and generation experiments. We then apply DBN-based pretraining to DNNs on the binarized MNIST dataset and analyze the influence of network depth, hidden-layer width, and training-set size.

This work aims to clarify when pretraining is beneficial, how it affects convergence and classification performance, and what its main limitations are compared with purely supervised training.

Implemented Models

In this project, we explored various models, including RBMs, DBNs, and DNNs, and evaluated their performance on two datasets : MNIST and Binary AlphaDigits.

1.1 Restricted Boltzmann Machines

Restricted Boltzmann Machines (RBMs) are a type of generative neural network model that falls under the category of energy-based models. RBMs consist of two layers of neurons : visible units and hidden units, with symmetric connections between them but no connections within the same layer. This restriction on connections gives RBMs their “restricted” nature.

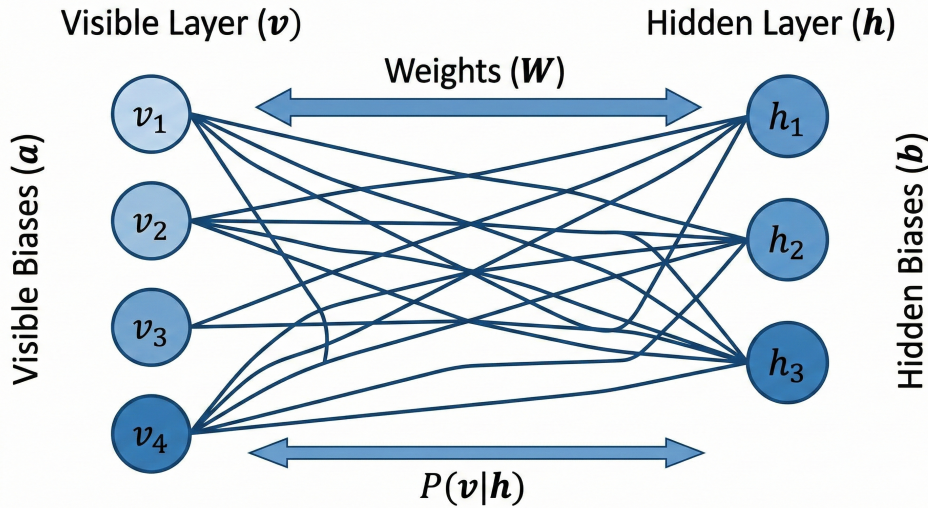


FIGURE 1.1 – Graphical description of a Restricted Boltzmann Machine (RBM).

The goal of an RBM is to model the probability distribution of the input data. The visible layer represents the observed data, while the hidden layer learns latent features that capture meaningful structures in the data.

Restricted Boltzmann Machine (RBM) Training Algorithm. The RBM is trained using the Contrastive Divergence algorithm, usually with one Gibbs sampling step (CD-1). The training procedure can be summarized as follows :

1. Initialize the weights W and the visible and hidden biases with small random values.
2. For each training example v :
 - (a) Sample the hidden units h from the conditional distribution $P(h | v)$.
 - (b) Reconstruct the visible units from $P(v | h)$.
 - (c) Update the weights and biases using the difference between the data statistics and the reconstruction statistics.

The Gibbs sampling equations are given by :

$$P(h_j = 1 | v) = \sigma \left(b_j + \sum_i W_{ij} v_i \right), \quad (1.1.1)$$

$$P(v_i = 1 | h) = \sigma \left(a_i + \sum_j W_{ij} h_j \right), \quad (1.1.2)$$

where $\sigma(x) = \frac{1}{1+e^{-x}}$ is the sigmoid activation function.

The parameter updates are computed using Contrastive Divergence :

$$\Delta W = \epsilon \left(\langle v h^T \rangle_{\text{data}} - \langle v h^T \rangle_{\text{model}} \right), \quad (1.1.3)$$

$$\Delta b = \epsilon \left(\langle v \rangle_{\text{data}} - \langle v \rangle_{\text{model}} \right), \quad (1.1.4)$$

$$\Delta c = \epsilon \left(\langle h \rangle_{\text{data}} - \langle h \rangle_{\text{model}} \right), \quad (1.1.5)$$

where :

- ϵ is the learning rate,
- $\langle \cdot \rangle_{\text{data}}$ denotes the average over the training data,
- $\langle \cdot \rangle_{\text{model}}$ denotes the average over the model distribution.

This training process enables the RBM to learn useful latent representations, which can later be used for reconstruction, generation, or as building blocks for deeper architectures.

1.2 Deep Belief Networks

A Deep Belief Network (DBN) is a type of generative deep neural network composed of multiple layers of latent variables, typically implemented as stacked Restricted Boltzmann Machines (RBMs). Each RBM is trained in an unsupervised, layer-wise manner to learn hierarchical representations of input data. Once pretraining is complete, the network can be fine-tuned using supervised learning techniques when applied to classification tasks. DBNs are useful for feature extraction, dimensionality reduction, and unsupervised learning.

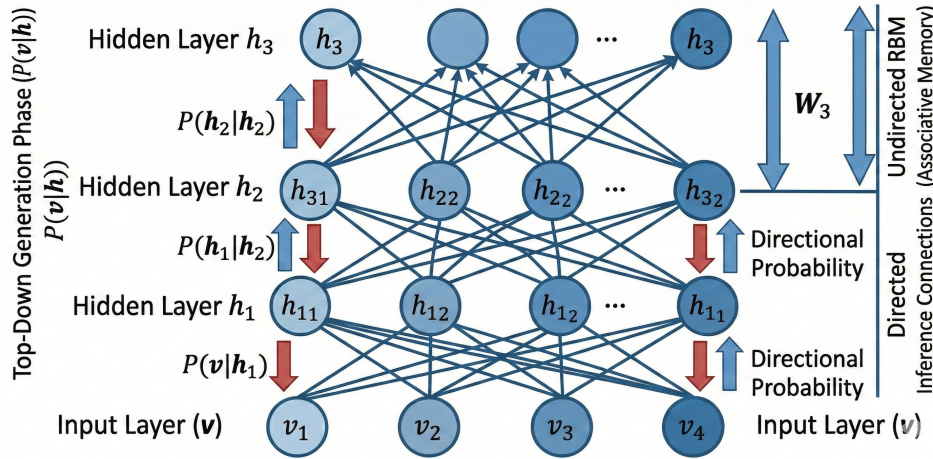


FIGURE 1.2 – Illustration of the basic structure of a Deep Belief Network (DBN).

A DBN is constructed by stacking several RBMs on top of each other. The first RBM is trained on the original input data, then the hidden activations of this RBM are used as the input for the next RBM, and the process is repeated layer by layer. In this way, the network gradually learns more abstract and hierarchical features.

The pretraining phase is unsupervised and follows a greedy layer-wise procedure :

- the first RBM learns low-level features from the input data,
- the second RBM learns higher-level representations from the hidden activations of the first one,
- additional RBMs can be stacked in the same manner.

Once all layers have been pretrained, the resulting DBN provides a meaningful initialization for a deep neural network. In addition, the DBN can also be interpreted as a generative model capable of producing samples through top-down generation.

1.3 Definition of Deep Neural Network (DNN)

A Deep Neural Network (DNN) is a class of artificial neural networks composed of multiple hidden layers between the input and output layers. Each hidden layer contains neurons that apply transformations to the input data, enabling the network to learn hierarchical representations. DNNs are trained using end-to-end backpropagation and optimization algorithms such as Stochastic Gradient Descent (SGD), Adam, or RMSprop.

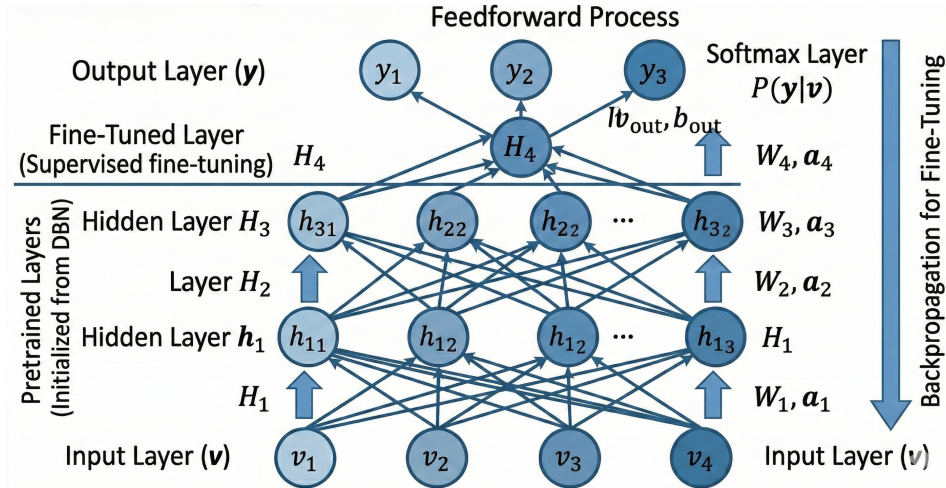


FIGURE 1.3 – Architecture of the Deep Neural Network (DNN) used for classification, where the hidden layers are initialized from DBN pretraining and the whole network is fine-tuned by backpropagation with a softmax output layer.

DNNs are widely used for supervised learning tasks, including image classification, natural language processing, speech recognition, and time-series forecasting. In this project, a DNN is constructed from a Deep Belief Network (DBN) by adding a Softmax activation function in the output layer, making it suitable for multi-class classification tasks.

To train the DNN, two approaches are explored :

- **Supervised Learning** : the DNN is trained directly using labeled data. During training, the network optimizes its parameters by minimizing the Cross-Entropy Loss through backpropagation.
- **Pretraining and Fine-Tuning** :
 - **Unsupervised Pretraining** : each hidden layer of the DBN is pretrained as an RBM. Each RBM learns to reconstruct its input and extract meaningful features. The output of one RBM is then used as the input of the next one.
 - **Fine-Tuning** : once all RBM layers have been pretrained, the entire network is fine-tuned using supervised backpropagation. This step adjusts all the parameters jointly in order to minimize classification errors.

This dual-phase approach leverages unsupervised pretraining to provide a strong initialization, followed by supervised fine-tuning to optimize the network for the final classification task.

Introduction to Datasets

2.1 Binary AlphaDigits Dataset

The Binary AlphaDigits dataset consists of images of handwritten characters encoded in binary format, where each pixel is represented as either 0 or 1. Each element in the dataset is structured as follows :

- **Image Representation** : each character is stored as a binary matrix, where pixels are either 0 (background) or 1 (foreground).
- **Accessing Data** : the dataset is available as a set of matrices. Each image can be accessed programmatically by loading the dataset into an appropriate data structure, such as a NumPy array in Python.
- **Labels** : each image is associated with a label corresponding to the character it represents, which makes the dataset suitable for both unsupervised and supervised learning experiments.

To facilitate data loading and visualization, a function named `lire_alpha_digit` was implemented. This function allows us to easily extract character images from the dataset and display them. An example of images from the Binary AlphaDigits dataset is shown in Figure 2.1.



FIGURE 2.1 – Example of images from the Binary AlphaDigits dataset.

2.2 MNIST Dataset

The MNIST dataset is a widely recognized benchmark in machine learning and computer vision. It contains a large collection of grayscale images representing handwritten digits from 0 to 9, each with a resolution of 28×28 pixels. Because of its simplicity and standard format, MNIST is commonly used to evaluate the performance of classification algorithms and deep learning models. An example of images from the MNIST dataset is shown in Figure 2.2.

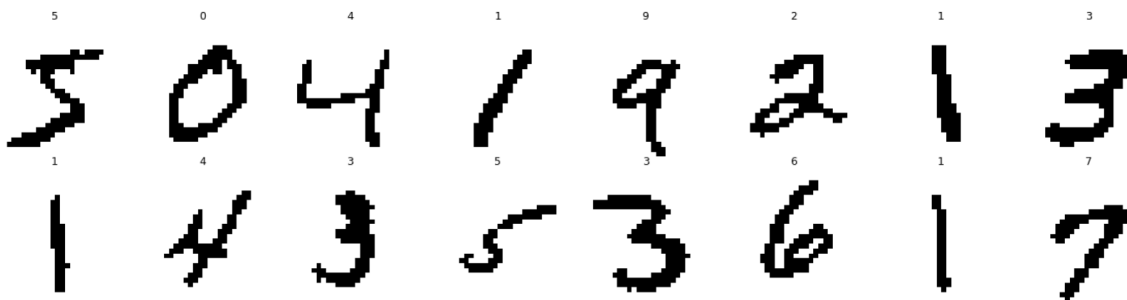


FIGURE 2.2 – Example of images from the MNIST dataset.

Experimental Protocol

This chapter summarizes the practical methodology used in the project. It focuses on the execution environment, the main training settings, the explored architectures, and the evaluation procedure.

3.1 Execution Environment

All experiments were conducted in a notebook-based environment with optional GPU acceleration. The main RBM/DBN/DNN pipeline was implemented **from scratch**, using NumPy on CPU and CuPy on GPU when CUDA was available. This design allows the same implementation to run on different hardware without modifying the code.

GPU acceleration is particularly beneficial during RBM/DBN pretraining and DNN fine-tuning, since these stages rely heavily on repeated matrix multiplications over mini-batches. For the bonus generative models (*VAE*, *GAN*, and *DDPM*), PyTorch was used separately.

3.2 Training Configuration

To ensure a fair comparison between *random initialization* and *DBN pretraining*, the supervised training procedure was kept identical across experiments. The only difference lies in the initialization of the hidden layers.

Unless stated otherwise, the default settings are :

- **Mini-batch size** : 64
- **RBM / DBN pretraining** : CD-1 with learning rate 0.1
- **DNN fine-tuning** : learning rate 0.1, 200 epochs
- **MNIST preprocessing** : binarization threshold = 127

Component	Default configuration
RBM training	CD-1, batch size 64
DBN pretraining	greedy layer-wise RBMs, lr = 0.1
DNN fine-tuning	softmax + cross-entropy, 200 epochs, lr = 0.1
MNIST preprocessing	binarization with threshold 127

TABLE 3.1 – Default settings used in the main experiments.

3.3 Architectures Explored

On **Binary AlphaDigits**, the main RBM configuration is [320, 200], with additional hidden sizes

50, 100, 200, 500

used for sensitivity analysis. For DBNs, the explored architectures are :

[320, 200], [320, 200, 100], [320, 200, 100, 50].

On **MNIST**, the baseline classifier is :

$$[784, 200, 200, 10].$$

Three factors were then investigated :

- **Depth** : $[784, 200, \dots, 200, 10]$ with $L \in \{1, 2, 3, 4, 5\}$,
- **Width** : $[784, h, h, 10]$ with $h \in \{100, 200, 300, 500, 700\}$,
- **Training-set size** : $n_{\text{train}} \in \{1000, 3000, 7000, 10000, 30000, 60000\}$ for the fixed architecture $[784, 200, 200, 10]$.

3.4 Evaluation Methodology

The models were assessed using both quantitative and qualitative criteria.

For RBMs and DBNs, training quality was monitored through the reconstruction Mean Squared Error :

$$\text{MSE}(x, \hat{x}) = \frac{1}{d} \sum_{j=1}^d (x_j - \hat{x}_j)^2.$$

For MNIST classification, the reported measures are accuracy and error rate :

$$\text{acc} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}\{\hat{y}_i = y_i\}, \quad \text{err} = 1 - \text{acc}.$$

Learning curves were also used to compare convergence and generalization between pretrained and randomly initialized networks.

In addition, the numerical results were complemented by qualitative diagnostics, including generated samples, softmax probability plots, misclassified examples, and confusion matrices.

3.5 How to Run the Code

The project was implemented as a notebook. To reproduce the experiments, the user only needs to place the `binaryalphadigs.mat` file in the working directory. The MNIST dataset is downloaded automatically by the notebook through a custom loader based on the Python standard library and NumPy, with a fallback to `torchvision` if direct download is unavailable.

The notebook should then be executed sequentially :

1. data loading and preprocessing,
2. RBM experiments on Binary AlphaDigits,
3. DBN experiments on Binary AlphaDigits,
4. MNIST pretraining and DNN fine-tuning,
5. bonus experiments (VAE, GAN, DDPM).

The main implementation uses `CuPy` when a CUDA-enabled GPU is available and falls back to `NumPy` otherwise. The bonus models are implemented with `PyTorch`. Some experiments are computationally expensive, so reduced runs with fewer epochs or smaller subsets can be used for quick verification before launching the full experiments.

Testing the Performance of Generative Models on the AlphaDigits Dataset

Before using generative models for pretraining deep classifiers on MNIST, we first validate our implementations on the Binary AlphaDigits dataset. This dataset is particularly well suited for such an analysis because the images are binary, low-dimensional, and visually easy to interpret. It therefore provides a convenient benchmark to evaluate both reconstruction and generation.

The purpose of this chapter is twofold : first, to verify that the implemented RBM and DBN learn meaningful binary representations ; second, to study how generation quality depends on model capacity, optimization settings, depth, and the number of learned classes. In most experiments, we focus on the letter **A**, which offers a controlled and visually clear setting.

4.1 Restricted Boltzmann Machine on Binary AlphaDigits

4.1.1 Generation and reconstruction on the letter A

We begin by training a Restricted Boltzmann Machine (RBM) on all available examples of the letter **A**. After training, new samples are generated by running Gibbs chains and collecting the visible states. The generated samples are shown in Figure 4.1.

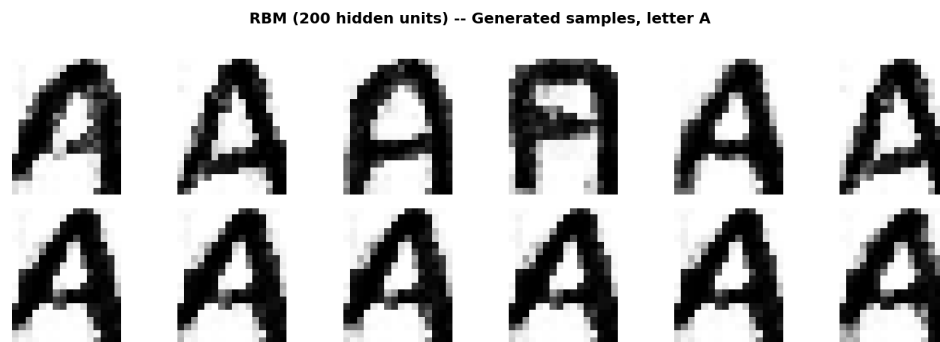


FIGURE 4.1 – Samples generated by the RBM on Binary AlphaDigits (letter A).

The generated images are clearly recognizable as handwritten **A**'s. The model successfully captures the two vertical strokes and the central bar, while still preserving some stochastic variability across samples. This already indicates that the RBM has learned the dominant structure of the class.

To confirm that this behavior is not merely qualitative, we also monitor the reconstruction Mean Squared Error (MSE) during training. The evolution of this quantity is shown in Figure 4.2.



FIGURE 4.2 – Reconstruction MSE of the RBM during training on the letter A.

The reconstruction error decreases smoothly and significantly, confirming that the RBM learns a meaningful internal representation of the data. Most of the improvement occurs during the early epochs, while the later phase mainly refines the reconstruction quality.

4.1.2 Effect of model capacity

We then study the impact of the number of hidden units on generation quality. The corresponding results are reported in Figure 4.3.

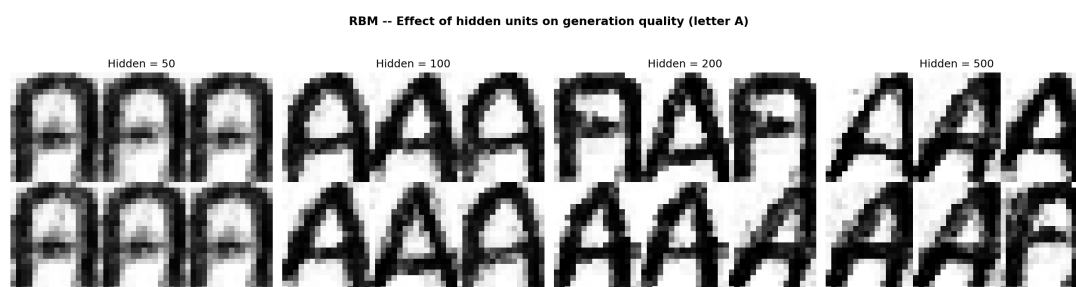


FIGURE 4.3 – Effect of the number of hidden units on RBM generation quality for the letter A.

The hidden dimension has a clear influence on the expressiveness of the model. With too few hidden units, the RBM captures the global outline of the character but remains limited in detail and flexibility. Increasing the hidden size improves the sharpness and consistency of the generated samples. However, beyond a moderate regime, the gain becomes less significant, which suggests that the complexity of a single handwritten character does not require very large hidden representations.

4.1.3 Effect of the learning rate

We also analyze the sensitivity of the RBM to the learning rate. Figure 4.4 shows representative generated samples obtained for different values.

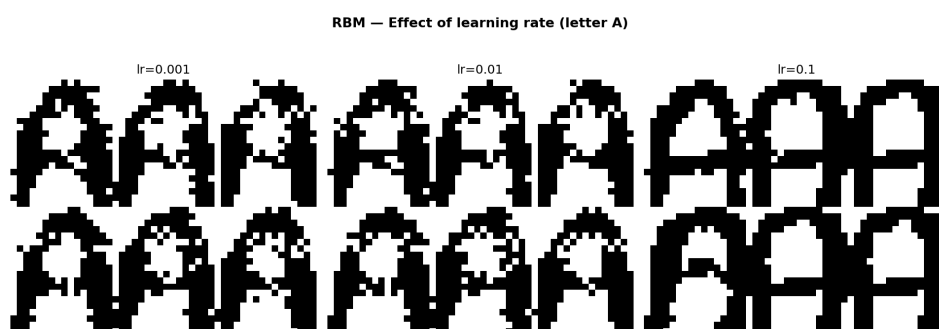


FIGURE 4.4 – Effect of the learning rate on RBM generation quality for the letter A.

A small learning rate leads to slow learning and blurred shapes, while a larger one produces sharper and more recognizable characters. In our experiments, the learning rate appears to be one of the most influential hyperparameters. This effect is confirmed in Figure 4.5, which compares the corresponding reconstruction quality.

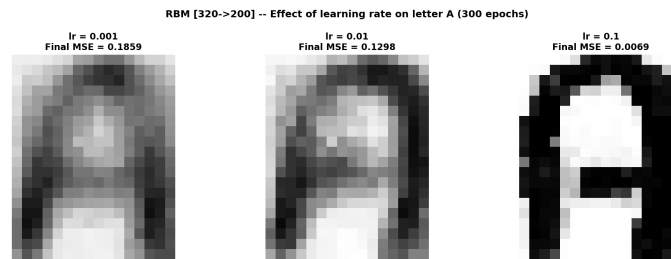


FIGURE 4.5 – Effect of the learning rate on reconstruction quality for the RBM trained on the letter A.

A better learning rate not only improves sample quality visually, but also leads to lower reconstruction error. This confirms that optimization settings play a central role even on a simple binary dataset.

4.1.4 Joint hyperparameter inspection

To better visualize the interaction between hyperparameters, we finally perform a small grid search over hidden size, learning rate, and sampling configuration.

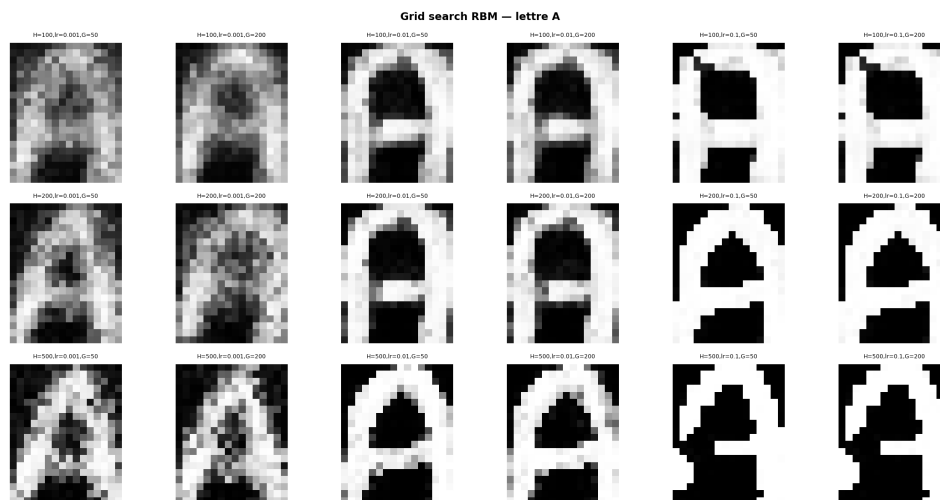


FIGURE 4.6 – Grid search for the RBM on the letter A : qualitative effect of hidden size, learning rate, and sampling configuration.

This summary confirms the previous trends : the learning rate has the strongest influence on the quality of the learned representation, whereas the number of hidden units mainly refines the richness and stability of the generated shapes.

4.2 Deep Belief Network on Binary AlphaDigits

We now consider Deep Belief Networks (DBNs), trained by greedy layer-wise pretraining. The goal is to determine whether stacking RBMs allows the model to learn more structured hierarchical features than a single RBM.

4.2.1 Layer-wise pretraining and generation

Each RBM layer of the DBN is trained sequentially on the stochastic activations of the previous layer. The reconstruction MSE for the different layers is shown in Figure 4.7.

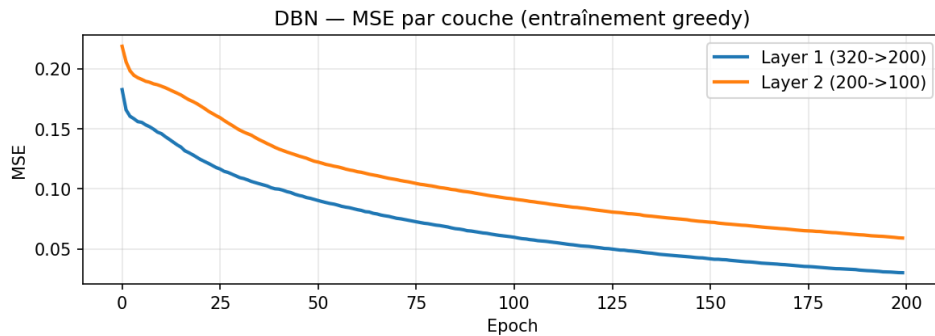


FIGURE 4.7 – Reconstruction MSE per layer during greedy DBN pretraining on Binary AlphaDigits.

Both layers exhibit a clear decrease in reconstruction error, indicating that the greedy training procedure functions correctly. As expected, the upper layer is harder to train and keeps a higher error level, but the downward trend confirms that it still extracts meaningful higher-level features.

After pretraining, new samples are obtained by sampling from the top-level RBM and propagating downward to the visible layer. Figure 4.8 reports typical generated examples for the letter **A**.

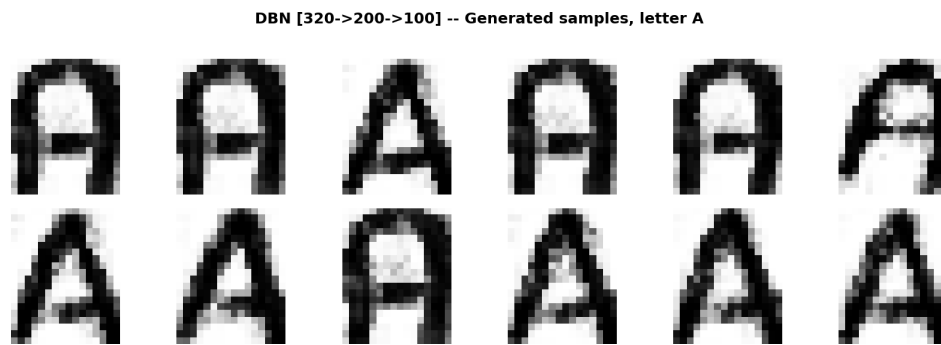


FIGURE 4.8 – Samples generated by the DBN on Binary AlphaDigits (letter A).

The generated letters remain clearly identifiable, and the DBN often produces more globally consistent shapes than the single RBM. This suggests that hierarchical feature extraction is indeed beneficial, even on a relatively simple binary dataset.

4.2.2 Effect of DBN depth

To assess the contribution of architectural depth, we compare DBNs with different numbers of hidden layers. The results are shown in Figure 4.9.

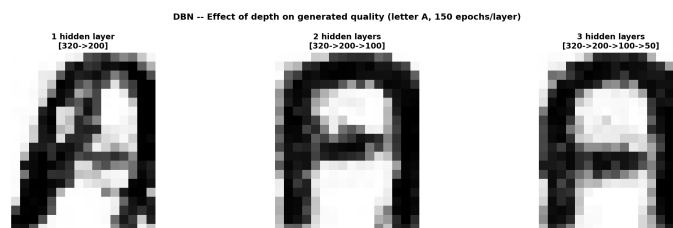


FIGURE 4.9 – Effect of DBN depth on generation quality for the letter A.

Adding a second hidden layer generally improves the global structure of the generated samples. However, increasing depth further does not systematically bring additional benefits. On this dataset,

moderately deep architectures appear sufficient, whereas deeper models may introduce extra smoothing or sampling difficulty without a clear qualitative gain.

4.3 Influence of the Number of Learned Classes

The previous experiments were mostly restricted to a single class. We now analyze what happens when the RBM or DBN is trained on several classes simultaneously. Since both models are unconditional generative models, increasing the number of classes makes the data distribution more multimodal and thus harder to model.

4.3.1 RBM

Figure 4.10 presents the effect of the number of learned classes for the RBM.

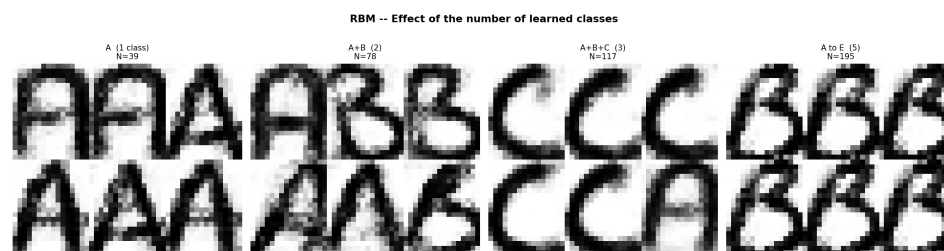


FIGURE 4.10 – Effect of the number of learned classes on RBM generation quality.

With a single class, the generated samples are sharp and class-consistent. As more classes are added, the learned distribution becomes increasingly complex and the generated patterns become less specific. The RBM tends to drift toward mixed or dominant modes, which reflects the difficulty of modeling a heterogeneous handwritten-character distribution with a shallow unconditional model.

4.3.2 DBN

The same experiment is carried out for DBNs, with the results shown in Figure 4.11.

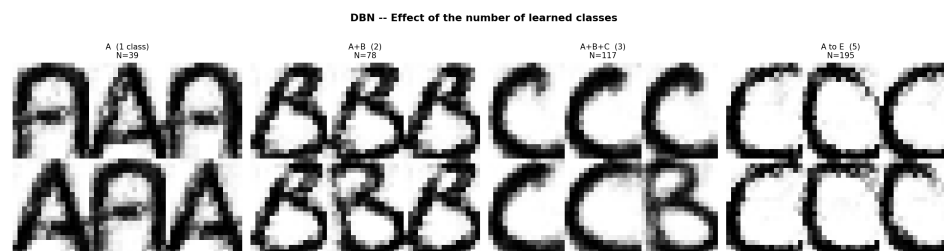


FIGURE 4.11 – Effect of the number of learned classes on DBN generation quality.

The overall trend is similar : sample quality degrades as the number of classes increases. However, the DBN tends to preserve more global structure than the RBM, especially when the number of classes remains small or moderate. This suggests that hierarchical representations partially mitigate the effect of increasing multimodality, although they do not completely remove it.

Application of Generative Models for Pretraining Deep Neural Networks on the MNIST Dataset

This chapter studies the use of generative pretraining for handwritten digit classification on MNIST. More precisely, it investigates whether RBM/DBN-based unsupervised pretraining provides a better initialization for deep neural networks than standard random initialization.

Two training strategies are compared throughout this chapter :

- **Random initialization** : the classifier is trained directly from scratch by supervised backpropagation ;
- **DBN pretraining + fine-tuning** : the hidden layers are first pretrained by greedy layer-wise RBM training, then the full network is fine-tuned with labeled data.

Since RBMs and DBNs rely on Bernoulli visible units, all MNIST images are binarized before pretraining.

5.1 Classification Pipeline

The MNIST pipeline used in the experiments is the following :

1. binarize the original grayscale images with threshold 127 and flatten them into 784-dimensional vectors ;
2. train a stack of RBMs greedily to form a DBN ;
3. add a softmax output layer on top of the pretrained hidden layers ;
4. fine-tune the complete network by supervised backpropagation using cross-entropy loss ;
5. evaluate the final classifier quantitatively and qualitatively.

5.2 Effect of Pretraining on Optimization

A first objective is to understand whether DBN pretraining improves optimization. This is especially important for sigmoid-based deep networks, which may become difficult to train from scratch when the architecture grows or when labeled data are limited.

Figure 5.1 compares the convergence of a network with architecture $[784, 200, 200, 10]$ under pretrained and randomly initialized settings.

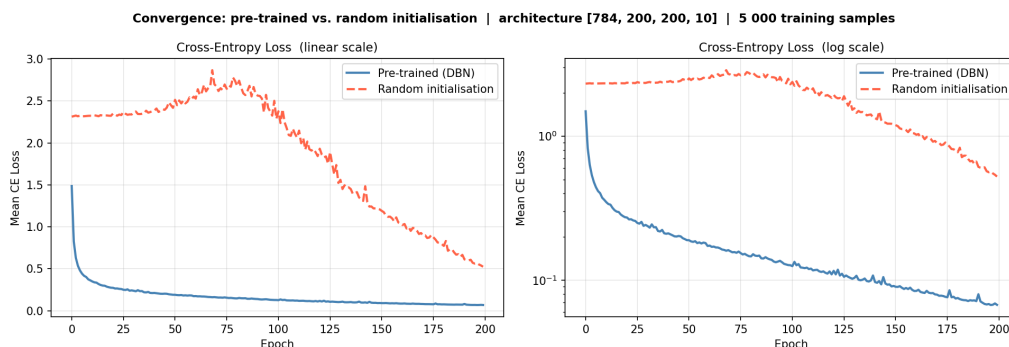


FIGURE 5.1 – Convergence comparison for $[784, 200, 200, 10]$: pretrained versus random initialization.

The pretrained model starts from a much better region of the parameter space and rapidly reaches low loss values, whereas the randomly initialized model learns more slowly and remains significantly worse over the whole trajectory.

This effect is confirmed by the learning curves in Figure 5.2, which report both cross-entropy loss and train/test classification error for $[784, 200, 200, 10]$ trained on 10,000 labeled samples.

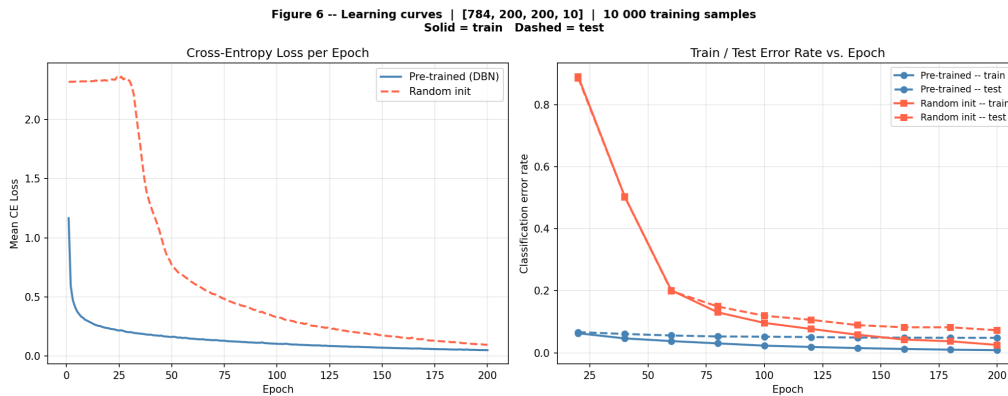


FIGURE 5.2 – Learning curves for $[784, 200, 200, 10]$ on 10,000 training samples : cross-entropy loss and train/test classification error.

Across the full optimization process, the pretrained network consistently achieves lower loss and lower error rates. This shows that unsupervised pretraining does not merely improve the final score : it also makes training faster and more stable.

5.3 Influence of Depth, Width, and Training-Set Size

Following the project requirements, three main factors were studied : the number of hidden layers, the hidden-layer width, and the number of labeled training samples. In all cases, pretrained and randomly initialized networks were trained under the same supervised conditions, so that the only difference comes from the initialization.

5.3.1 Effect of Depth

We first vary the number of hidden layers L and consider architectures of the form

$$[784, \underbrace{200, \dots, 200}_{L \text{ times}}, 10].$$

The corresponding train and test error rates are shown in Figure 5.3.

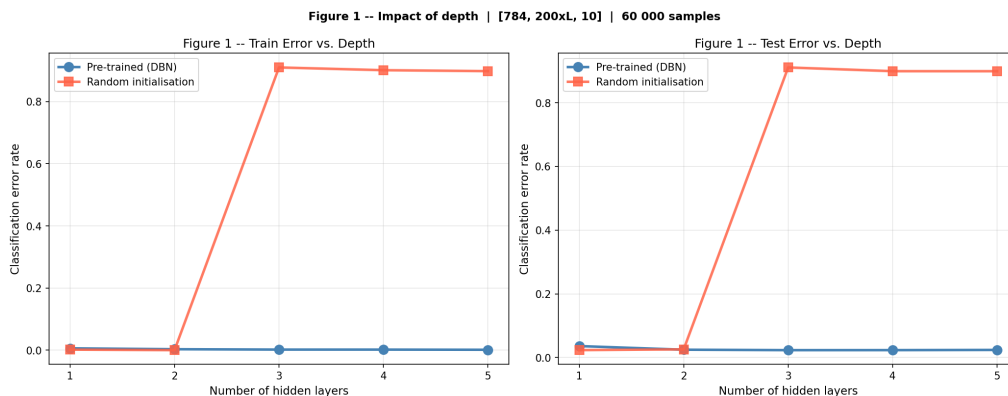


FIGURE 5.3 – Impact of network depth on MNIST classification error (train and test).

The pretrained networks remain stable as depth increases and keep a low test error across all tested configurations. In contrast, the randomly initialized networks become highly unstable beyond shallow architectures, and their performance may collapse toward near-chance behavior. This is one of the strongest results of the chapter : DBN pretraining is particularly beneficial when optimization becomes difficult because of network depth.

5.3.2 Effect of Width

We then consider two-hidden-layer networks of the form

$$[784, h, h, 10],$$

for increasing values of h . The results are reported in Figure 5.4.

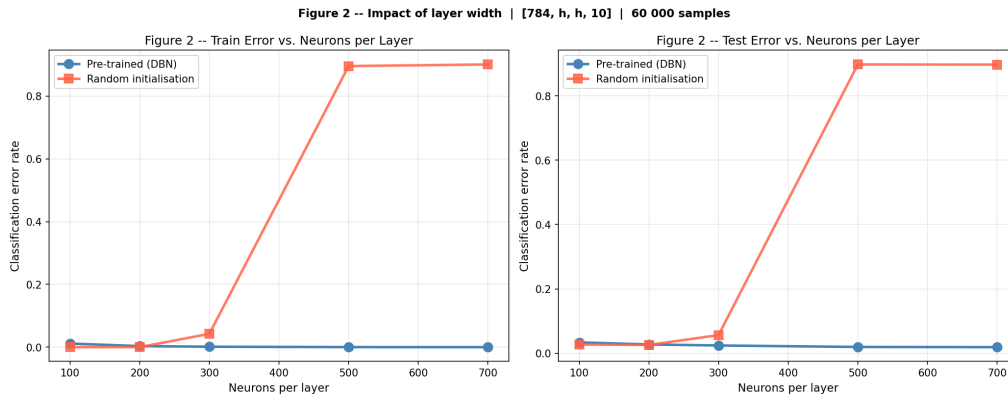


FIGURE 5.4 – Impact of hidden-layer width on MNIST classification error (train and test).

The pretrained DNN remains robust as width increases and can make effective use of larger hidden representations. By contrast, the randomly initialized DNN becomes unstable for sufficiently wide networks and fails to exploit the additional capacity. This shows that the benefit of pretraining is not limited to depth : it also extends to wider architectures.

5.3.3 Effect of Training-Set Size

Finally, we fix the architecture to

$$[784, 200, 200, 10]$$

and vary the number of labeled training samples :

$$n_{\text{train}} \in \{1000, 3000, 7000, 10000, 30000, 60000\}.$$

The results are shown in Figure 5.5.

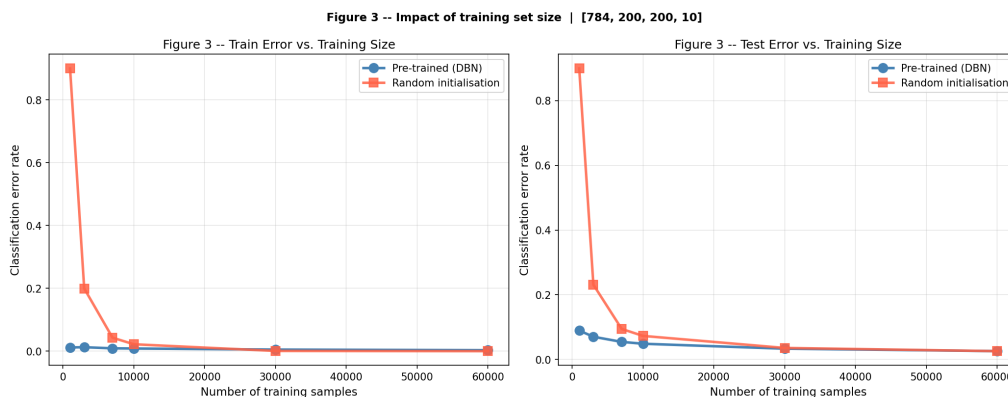


FIGURE 5.5 – Impact of training-set size on MNIST classification error (train and test).

The gain from pretraining is strongest in the low-data regime. When only a limited number of labeled samples is available, the pretrained DNN achieves a clearly lower test error than the randomly initialized baseline. As the amount of supervision increases, the performance gap narrows, and both approaches become closer on the full MNIST training set.

Overall, these three experiments lead to the same conclusion : DBN pretraining is most useful when the supervised learning problem is more challenging, either because the architecture is harder to optimize or because the number of labels is small.

5.4 Qualitative Analysis of the Pretrained Classifier

To complement the quantitative results, we analyze the behavior of the pretrained classifier on individual test examples using softmax probabilities, misclassified images, and the confusion matrix.

5.4.1 Softmax Confidence

Figure 5.6 displays the softmax output probabilities for selected MNIST test samples.

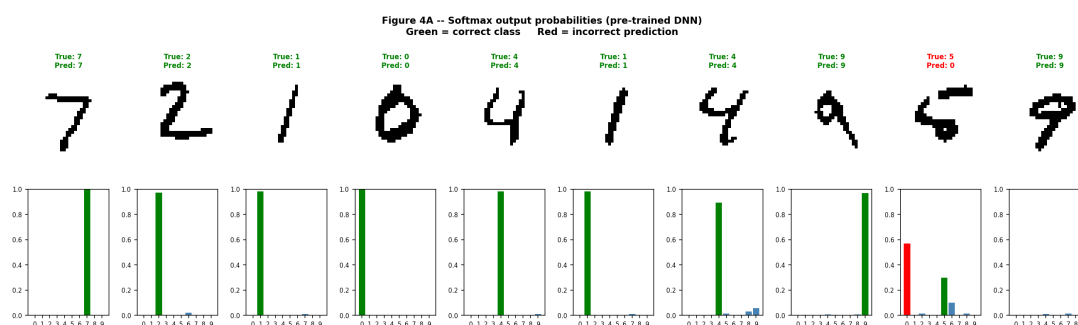


FIGURE 5.6 – Softmax output probabilities for selected MNIST test examples using the pretrained DNN.

Correct predictions are generally associated with highly peaked distributions, which indicates strong confidence. For more ambiguous samples, the probability mass is shared across visually similar classes. Even in misclassified cases, the output often remains interpretable and reflects genuine visual ambiguity rather than arbitrary behavior.

5.4.2 Misclassified Examples

Figure 5.7 shows a subset of misclassified test images together with their true and predicted labels.

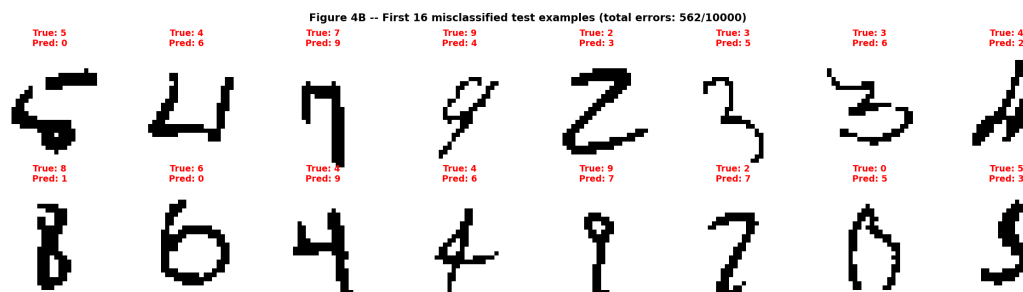


FIGURE 5.7 – First misclassified MNIST test examples for the pretrained DNN.

Most of these errors are visually understandable. They often correspond to unusual handwriting, incomplete strokes, or digits whose shapes lie between several plausible classes. This confirms that the remaining errors are concentrated on genuinely difficult examples.

5.4.3 Confusion Matrix

A more global view is provided by the confusion matrix shown in Figure 5.8.

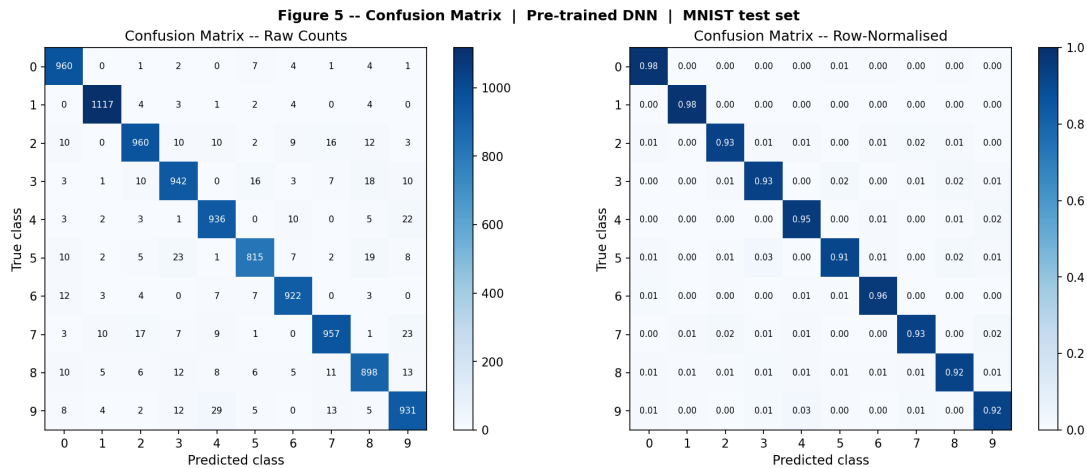


FIGURE 5.8 – Confusion matrix on the MNIST test set for the pretrained DNN (raw counts and row-normalized).

The diagonal is dominant, indicating strong overall classification performance. The most frequent confusions correspond to visually similar digits such as 4/9, 5/3, 2/7, and 6/0, which is consistent with the softmax analysis and the displayed misclassified examples.

5.5 Best Final Configuration

In addition to the controlled experiments on depth, width, and training-set size, a broader search over pretrained architectures was carried out in order to identify the strongest classifier obtained in this project.

Among the tested candidates, the best result was achieved by a **DBN-pretrained DNN** with architecture

$$[784, 700, 700, 10].$$

This configuration reached a test accuracy of approximately

$$\text{acc}_{\text{test}} \approx 0.9827,$$

which corresponds to a test error rate of

$$\text{err}_{\text{test}} \approx 0.0173.$$

Therefore, the best-performing classifier obtained in this work is the pretrained architecture $[784, 700, 700, 10]$, which provides the strongest MNIST test performance among the tested configurations.

Bonus : Testing the Performance of Generative Models on the MNIST Dataset

In this bonus section, we extend the study beyond RBM/DBN pretraining and compare several generative models directly on the MNIST dataset from a **qualitative generation** perspective. The goal is not to build a perfectly standardized benchmark, but rather to visually compare the types of samples produced by different families of generative models under practical training conditions.

The comparison includes :

- a **Restricted Boltzmann Machine (RBM)**,
- a **Deep Belief Network (DBN)**,
- a **Variational Autoencoder (VAE)**,
- a **Generative Adversarial Network (GAN)**,
- and a **Denoising Diffusion Probabilistic Model (DDPM)**.

We focus on three main qualitative criteria :

- **readability** : whether the generated image is clearly recognizable as a digit,
- **sharpness** : whether strokes are well defined or overly blurred,
- **diversity** : whether the model produces different digit shapes and styles.

6.1 Global Visual Comparison

Figure 6.1 provides a direct visual comparison of the generated samples for all the considered models.

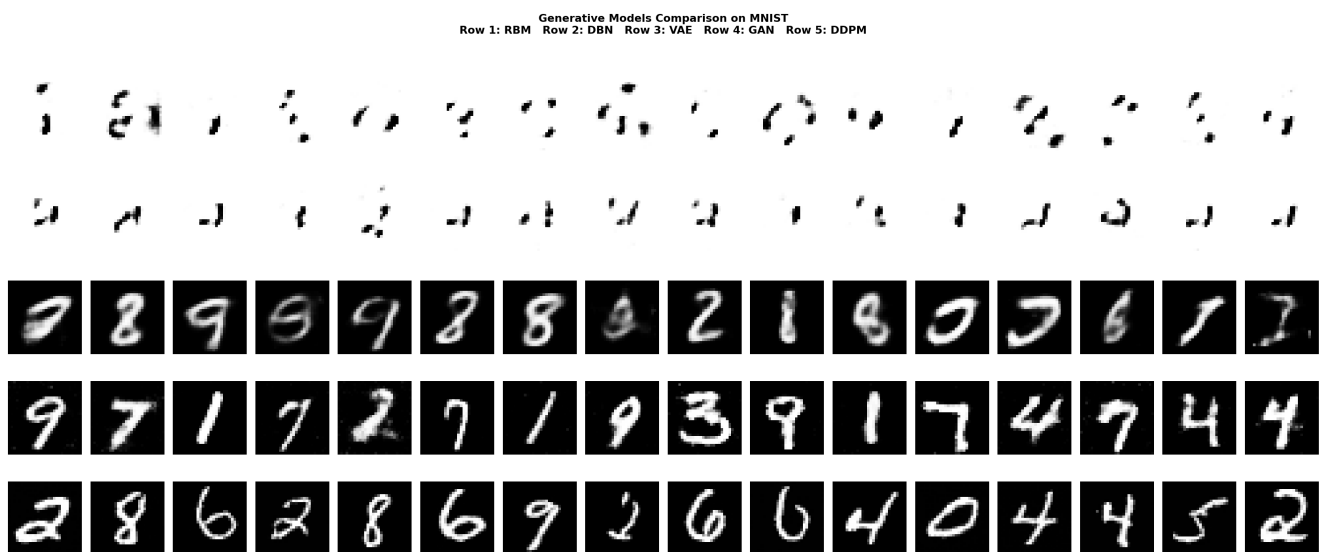


FIGURE 6.1 – Visual comparison of generated MNIST samples. From top to bottom : RBM, DBN, VAE, GAN, and DDPM.

A clear hierarchy emerges from this figure. The RBM produces the weakest samples, the DBN improves global structure only slightly, the VAE yields coherent but smooth digits, the GAN produces

sharper outputs, and the DDPM provides the most consistently readable and visually realistic samples among the tested methods.

This comparison is particularly informative in the context of the whole project : while RBMs and DBNs are useful for *representation learning* and *pretraining*, they are not the strongest models for *direct image generation* on a dataset as rich as MNIST.

6.2 RBM

We first consider the RBM trained directly on binarized MNIST.

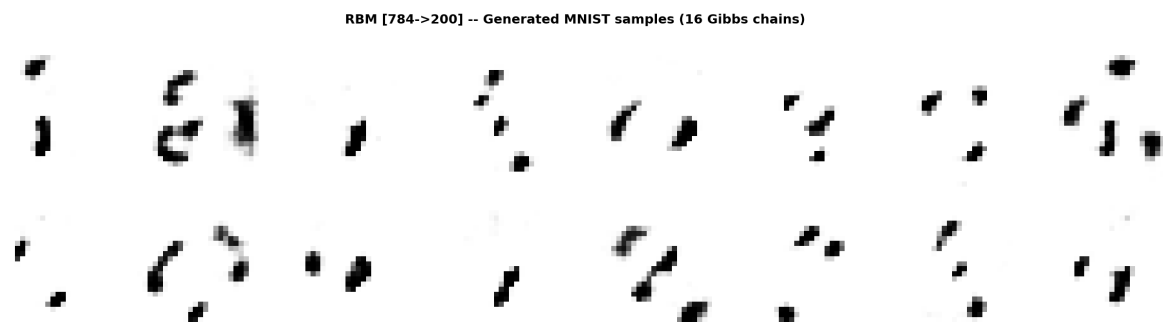


FIGURE 6.2 – Samples generated by an RBM trained on MNIST.

The generated samples are mostly composed of isolated strokes or small digit fragments. In some cases, local structures reminiscent of handwritten digits can be observed, but the global digit shape is rarely recovered. The samples are therefore difficult to interpret and exhibit limited realism.

This result is not surprising. The RBM is a shallow energy-based model, and generation relies on Gibbs sampling in a high-dimensional space. While this can work reasonably well on very simple binary datasets such as single-class AlphaDigits, it becomes much more challenging on a multimodal dataset like MNIST, where the model must account for multiple digit classes and large intra-class variability.

Therefore, on MNIST, the RBM appears more useful as a building block for pretraining than as a standalone generative model.

6.3 DBN

We then consider generation with a DBN obtained by stacking pretrained RBMs.

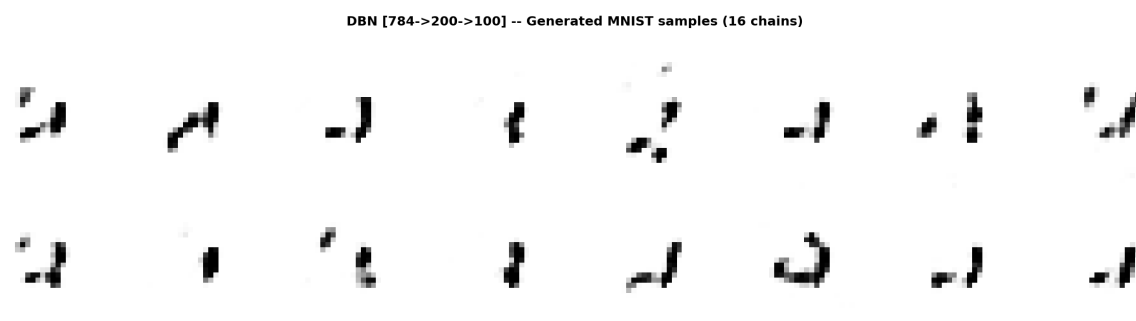


FIGURE 6.3 – Samples generated by a DBN trained on MNIST.

Compared with the RBM, the DBN produces samples that are slightly more structured. Some images resemble digits more clearly than the RBM outputs, and the model is able to capture larger-scale stroke organization. However, the generations remain weak overall : many samples are still fragmented, incomplete, or overly faint.

This result is consistent with the observations made in the main chapters. On MNIST, DBNs are especially valuable as *unsupervised initializers* for deep classifiers, but their direct generative quality remains limited in our implementation. In other words, a DBN can learn useful internal representations without necessarily being a strong sampler.

This contrast is itself an interesting outcome of the project :

good unsupervised pretraining does not automatically imply strong unconditional image generation.

6.4 VAE

We next evaluate the Variational Autoencoder (VAE).

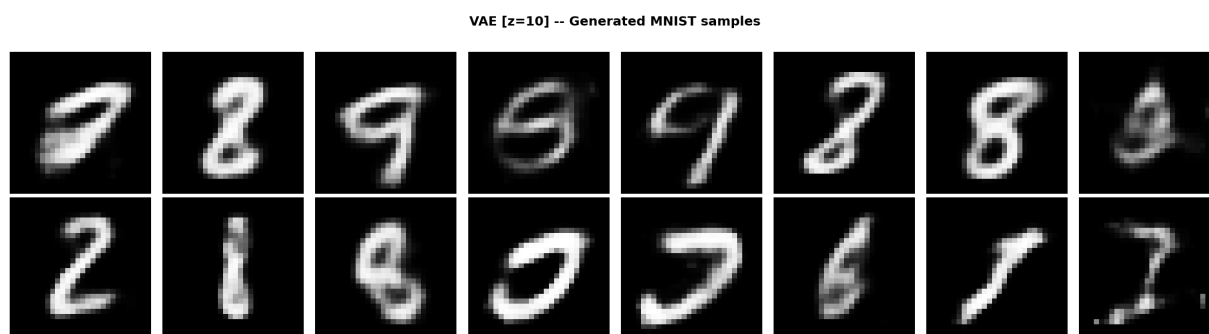


FIGURE 6.4 – Samples generated by a VAE trained on MNIST.

The VAE produces clearly recognizable digits. The samples are coherent, and the model covers multiple digit identities and writing styles. This shows that the learned latent representation captures the overall structure of the MNIST distribution reasonably well.

The main limitation is the usual one observed with VAEs : the generated digits tend to be slightly smooth or blurry. This is a common visual signature of latent-variable models trained with a reconstruction objective and latent regularization. Despite this softness, the outputs are significantly more realistic than those obtained with RBM and DBN generation.

Thus, in our experiments, the VAE provides a good balance between readability and diversity, even if it does not produce the sharpest images.

6.5 GAN

We now turn to the GAN-based model.

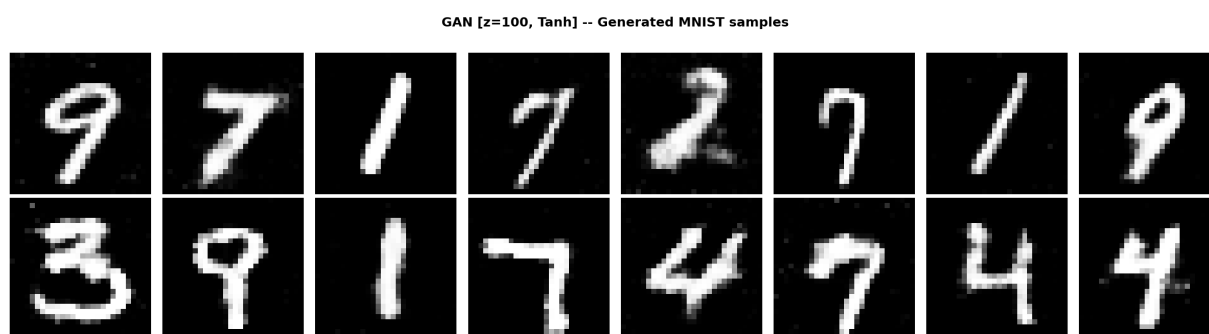


FIGURE 6.5 – Samples generated by a GAN trained on MNIST.

The GAN produces samples that are visually sharper than those of the VAE. Strokes are better defined, the digit contours are more contrasted, and many images appear immediately plausible as handwritten MNIST digits. In terms of visual realism, the GAN clearly outperforms the RBM and DBN and is often more convincing than the VAE.

Some limitations nevertheless remain. A few samples are distorted or noisy, and the diversity of the generated digits may be less balanced than with latent-variable models. This is consistent with the well-known training difficulties of GANs, including instability and possible mode concentration.

Still, within our bonus experiments, the GAN achieves a strong qualitative level and represents a significant improvement over classical RBM/DBN generation.

6.6 DDPM

Finally, we evaluate a Denoising Diffusion Probabilistic Model (DDPM), implemented with a U-Net-based noise predictor.

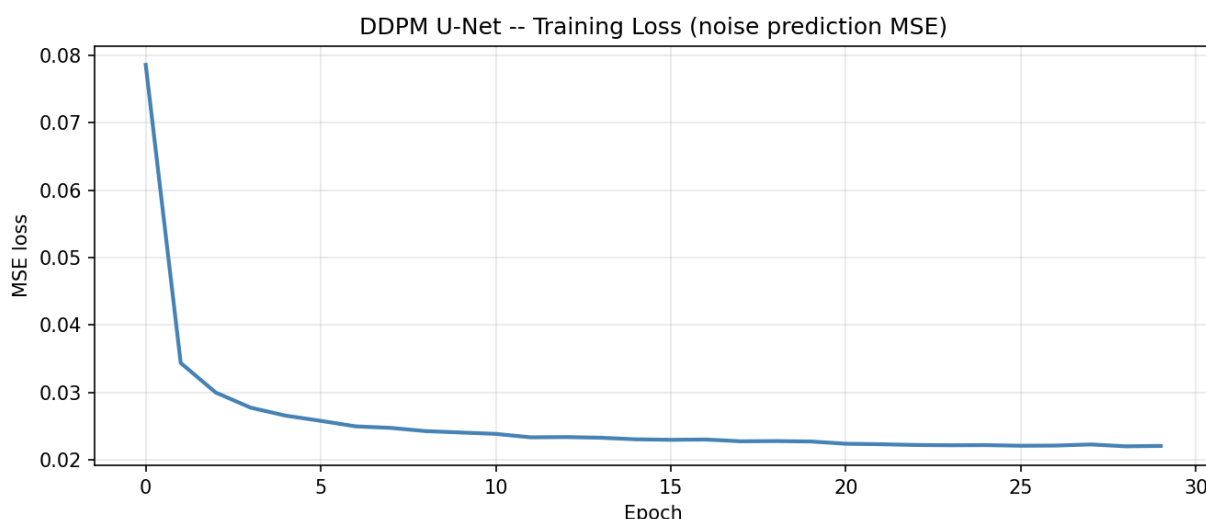


FIGURE 6.6 – Training loss of the DDPM noise predictor on MNIST.

Figure 6.6 shows a stable and monotonic decrease of the noise-prediction loss during training, which indicates that the reverse denoising model is learning appropriately.

The resulting generated samples are shown in Figure 6.7.

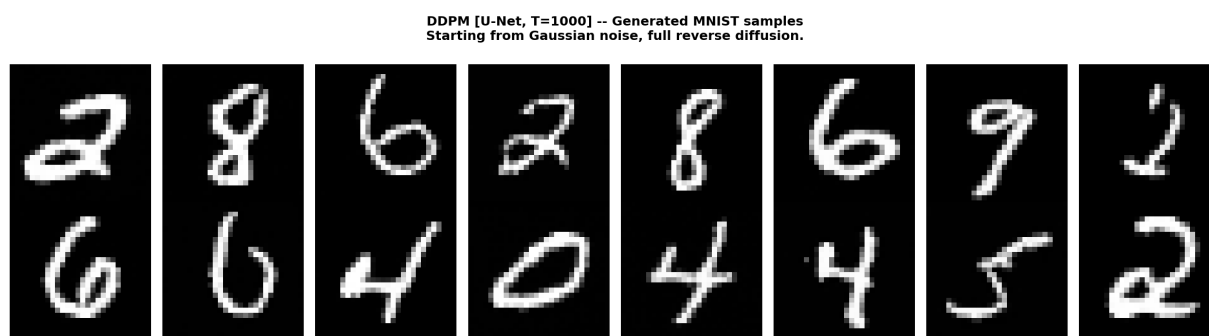


FIGURE 6.7 – Samples generated by a DDPM trained on MNIST.

Among all models tested in this bonus section, the DDPM produces some of the most convincing samples. The generated digits are generally readable, structurally coherent, and visually clean. In addition, the diversity across samples remains good, with multiple digit classes and handwriting styles represented.

Compared with the GAN, the DDPM often yields more stable visual quality across samples, while preserving high readability. Compared with the VAE, it produces sharper and more realistic digits. The contrast with RBM and DBN is particularly striking.

These results confirm the strength of diffusion models for image generation, even on a relatively small benchmark such as MNIST.

Conclusion

In this project, we studied the use of generative models for representation learning, image generation, and neural network initialization. We first implemented and analyzed Restricted Boltzmann Machines (RBMs), Deep Belief Networks (DBNs), and Deep Neural Networks (DNNs) from scratch, and evaluated their behavior on the Binary AlphaDigits and MNIST datasets.

The experiments on Binary AlphaDigits showed that RBMs and DBNs are able to learn meaningful binary structures and generate recognizable handwritten characters. These experiments also highlighted the importance of key hyperparameters such as the learning rate, the hidden dimension, and the model depth.

On MNIST, the main result of the project is that DBN pretraining provides a clear advantage for supervised classification. Compared with random initialization, pretrained networks converge faster, remain more stable for deeper and wider architectures, and perform better when the amount of labeled data is limited. Qualitative analyses based on softmax probabilities, misclassified examples, and confusion matrices further confirmed the relevance of the learned representations.

Finally, the bonus comparison between RBM, DBN, VAE, GAN, and DDPM showed that generative models do not all serve the same purpose equally well. While RBMs and DBNs are especially useful for unsupervised pretraining, more modern models such as VAEs, GANs, and diffusion models achieve much stronger image generation quality on MNIST.

Overall, this project illustrates both the historical importance and the practical interest of generative pretraining, while also situating it in the broader landscape of modern generative modeling.